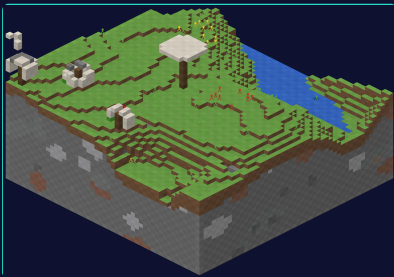


Persistent 3D World Models

Generate the world in 3D — then put a camera *inside* it.



generated 3D voxel world

render



rendered RGB

What is a world model?

A neural network that **simulates an environment**:



State of the art (2024):

Oasis (Decart / Etched) —
real-time, playable Minecraft.

Genie (Google DeepMind) —
generates whole 2D worlds from a
prompt.

Stunning — but they share one deep flaw.



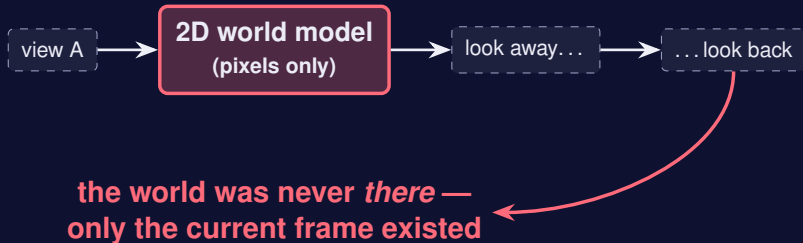
Oasis (Decart/Etched)



Genie (Google DeepMind)

**both are 2D — no
persistent world**

The bottleneck: 2D models have no world



They generate **2D pixels**, frame-by-frame. There is no underlying world.
Look away and back $\Rightarrow$ that region is **re-hallucinated**, different every time.
No persistence. No consistent geometry.

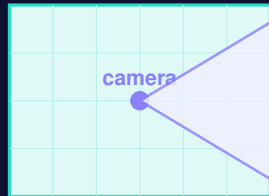
Our thesis: persistence is a *representation* problem

If the world only lives in the current 2D frame, it *cannot* persist.

So we flip the formulation:

1. Generate a real **3D** representation of the world.
2. Place a camera *inside* it and render what it sees.

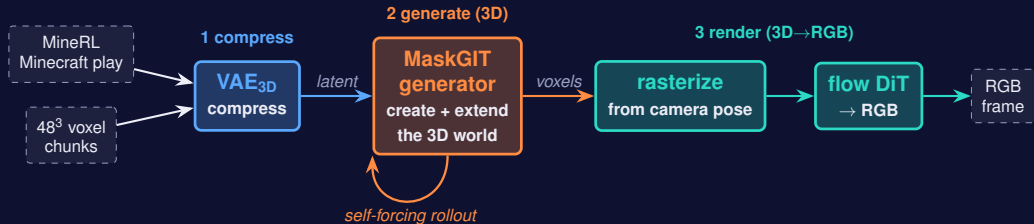
Persistence is now free — the geometry is literally stored. Look away and back: *same world*, because the world is an actual object, not a frame.



persistent 3D voxel world

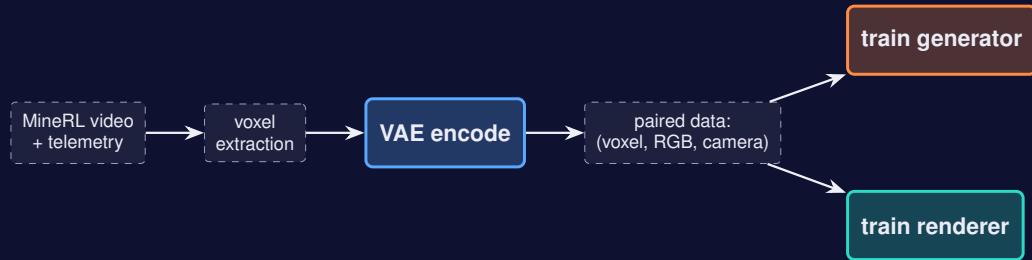
the camera moves *through* a fixed object

System architecture



- 1 **VAE_{3D}** compresses voxels to a latent 2 **MaskGIT** generates & extends the world as the agent moves 3 **Renderer** rasterizes the 3D from the camera, diffusion → photorealistic RGB

Research: data & training

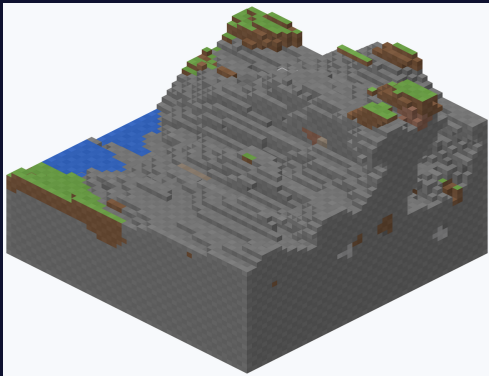


Data: VPT / MineRL *treechop* human play.
Each episode $\rightarrow$ 48^3 block grids $\rightarrow$ VAE latents.

Generator: predict / extend the voxel world.
Renderer: learn voxels $\rightarrow$ RGB from paired frames.

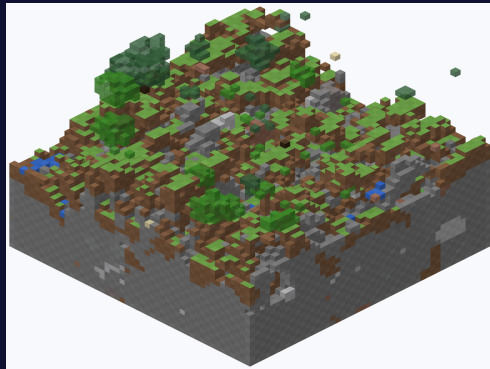
The core research problem: rollout collapse

Naive autoregression collapses



after 64 self-steps → grey stone mush

With self-forcing



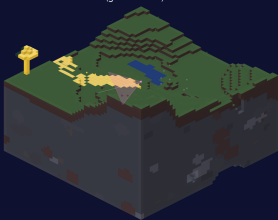
stays diverse & green across the rollout

Self-forcing (Dagger-style): during training, feed the model its *own* generations as context and force it to recover toward ground truth. Errors stop compounding ⇒ stable long rollouts. (distinct block types retained: 14 → 21; non-air acc 0.83 → 0.92).

Renderer & persistence

A camera *inside* the 3D world; **gold** = the voxels it can see $\rightarrow$ render $\approx$ ground truth.

A voxel world + camera (gold = visible)



B ground-truth RGB



C our 3D->RGB render



A voxel world + camera + visible voxels (gold) | B ground-truth RGB | C our render (HF detail within $\sim 1\%$ of GT)

Persistence: turn the camera and the world stays *exactly* consistent — we move a camera through a *fixed* 3D object, not re-generate pixels.

[play `figs/persist_viz_turn.mp4` — the world is identical as the camera yaws]

Demo: walk 15 blocks out and back → same world

controls

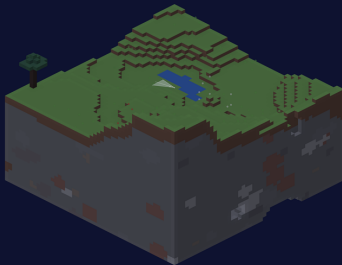
generated 3D world + camera

first-person render



BACK AT CENTER

+X=W -X=S +Z=D -Z=A



WASD (from motion) | generated 3D world + camera | first-person render — agent walks 15 blocks out-and-back in every direction

On every return to center the world is **100% bit-identical** — a model-generated 3D world that *remembers* what it generated.

[play figs/persist_demo/final_rollout_persistence.mp4]

Use cases & impact

Embodied AI / RL

stable, replayable 3D sims
+ free ground-truth state

Multi-agent worlds

one shared, consistent
3D world for many agents

Games / content

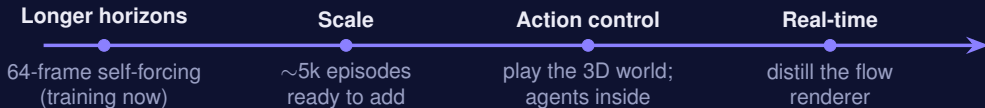
explorable 3D levels
you can walk around

Synthetic data

any camera, any lighting
controllable generation

A consistent 3D substrate is what 2D video models fundamentally cannot provide — and it's the prerequisite for **multi-agent** simulated worlds.

What's next



World models don't have to be flat videos that forget themselves.

Build the world in 3D. Put the camera inside. Persistence comes for free.



Thank you · CS153 Final Project